Laboratorio de Analítica de Datos - Sede Bogotá -

UNIVERSIDAD NACIONAL DE COLOMBIA

Ingeniería de Prompts II: Estrategias para problemas complejos

Estructura y planeación

PhD. Oscar Alberto Rodríguez Meléndez
Universidad Nacional de Colombia

2026·II

DataLab UNAL

Contenido y recorrido

Fundamentos y estrategias

1. Problemas complejos y perspectiva.
2. Step-Back y Analogical Prompting.
3. Plan-and-Solve y Decomposed Prompting.

Exploración y aplicación

4. Tree-of-Thought y evaluación.
5. Skeleton-of-Thought y síntesis.
6. Chat de consola en Python con dos modos.

Idea clave

El caso de uso depende de la tarea, no importan los nombres, importa el objetivo

Objetivos de aprendizaje

- Distinguir problemas que requieren **planificación**, **descomposición** o **exploración**.
- Aplicar Step-Back, Analogical Prompting, Plan-and-Solve, Decomposed Prompting y Tree-of-Thought.
- Diferenciar **rutas alternativas** de una **estructura global de respuesta**.
- Elegir estrategias según la estructura real del problema y verificar sus resultados.
- Dirigir un agente de código para elaborar ejercicios **en Python** y construir un chat de consola.

Idea central

Una tarea puede estar bien definida y aun así necesitar una estrategia explícita antes de producir una respuesta útil.

De dónde venimos

En la sesión anterior

Definimos tareas mediante contexto, criterios, demostraciones y formato; revisamos zero-shot, one-shot, few-shot, razonamiento guiado, Least-to-Most y evaluación.



Idea clave

Ahora estudiaremos cómo elegir y coordinar estrategias para tareas abiertas, no cómo escribir prompts más largos.

¿Cómo resolvemos un problema?



La nueva pregunta

Una tarea puede estar perfectamente especificada y, sin embargo,

no tener una ruta de resolución evidente.

Pregunta guía

¿Qué hacemos cuando primero debemos decidir **cómo abordar** el problema antes de intentar resolverlo?

Tarea directa vs. problema abierto

Laboratorio de Analítica de Datos - Sede Bogotá -

UNIVERSIDAD NACIONAL DE COLOMBIA

Tarea directa

Clasifica un mensaje en **académico, financiero o técnico**.

- Salida delimitada.
- Criterios observables.
- Pocas rutas de resolución.

Problema abierto

Diseña un sistema que reciba, clasifique y priorice solicitudes.

- Varias arquitecturas posibles.
- Decisiones sobre datos y errores.
- Restricciones de recursos y tiempo.

Idea clave

La calidad depende también de cómo organizamos la búsqueda y comprobación de una solución.

DataLab UNAL

Caso transversal: solicitudes institucionales

Situación

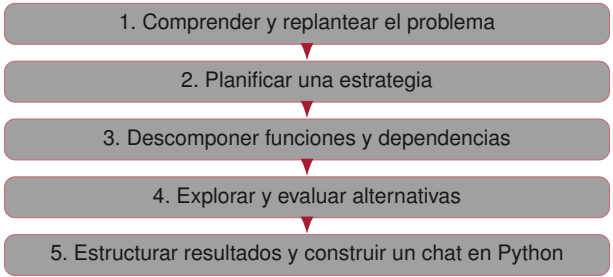
Una institución recibe mensajes académicos, financieros y técnicos. Necesita un prototipo de clasificación y atención, usando solamente datos ficticios.

ID	Solicitud	Etiqueta de referencia
S01	No puedo descargar mi certificado.	Técnico
S02	Necesito conocer las fechas de matrícula.	Académico
S03	Mi pago no aparece registrado.	Financiero
S04	La plataforma cierra mi sesión.	Técnico
S05	Quiero solicitar una beca.	Financiero
S06	No sé dónde consultar mi solicitud.	Revisión manual

Precaución

Estas etiquetas son criterios didácticos fijados para el ejercicio; no demuestran que un clasificador real sea fiable.

Mapa de la sesión



Idea clave

Cada actividad añade un artefacto reutilizable: principios, plan, módulos, decisiones e informe.

El riesgo de responder demasiado pronto

Laboratorio de Analítica de Datos - Sede Bogotá -

UNIVERSIDAD NACIONAL DE COLOMBIA

Prompt directo

«Crea un programa que clasifique solicitudes y recomiende qué área debe atenderlas».

- El agente puede elegir categorías y reglas sin consultarlas.
- Puede empezar a programar sin comprobar los datos disponibles.
- Una primera solución plausible puede ocultar alternativas mejores.

Idea clave

A veces conviene cambiar el **punto de partida** antes de cambiar la solución.

DataLab UNAL

Step-Back Prompting

Idea central de la técnica

Antes de resolver el caso puntual, se identifican **principios, dimensiones o estructuras generales** del problema; luego se aplican al caso concreto.

- Permite evitar respuestas estrechas o basadas en el primer supuesto.
- La abstracción debe regresar a las restricciones y datos disponibles.
- No convierte principios generales en evidencia específica del caso.

Fuente: Zheng et al., Take a Step Back (2023)

Step-Back: pasar del caso a los principios



Pregunta de abstracción

¿Qué caracteriza a un **buen sistema de clasificación de solicitudes**, con independencia de la herramienta utilizada?

Idea clave

Después de abstraer, debemos comprobar que cada principio sea aplicable a los mensajes y recursos del ejercicio.

Step-Back aplicado a las solicitudes

Fase 1: dar un paso atrás

Antes de programar, identifica principios para clasificar solicitudes: categorías exhaustivas, manejo de ambigüedad, trazabilidad de decisiones y facilidad de revisión. Indica cómo comprobarlos.

Fase 2: volver al caso

Utiliza los seis mensajes S01–S06. Propón criterios observables y una solución inicial con Python. Para cada criterio indica si está respaldado por los ejemplos o si necesita validación.

Idea clave

No se trata de pedir «más teoría», sino de elegir un marco adecuado antes de implementar.

Prompt directo vs. Step-Back

	Directo	Con Step-Back
Punto de partida	«Clasifica estos mensajes».	«Define qué principios hacen fiable la clasificación».
Riesgo	Reglas y supuestos implícitos.	Abstracciones sin relación con los datos.
Control	Revisar etiquetas finales.	Contrastar principios, reglas y etiquetas.
Salida esperada	Un prototipo inicial.	Un prototipo y criterios para revisarlo.

Interpretación

No hay una técnica ganadora por definición: compare la utilidad y los errores de ambas ejecuciones.

Actividad 1: construir después de abstraer

En Kiro, Codex, Claude Code o equivalente — 10 minutos

1) Pide principios generales para clasificar mensajes.
2) Revisa y selecciona los aplicables a S01–S06.
3) Solicita **solo Python** para una primera versión del clasificador, sin servicios externos.
4) Ejecuta y revisa especialmente S06.

Producto y comprobación

Una función de Python que devuelva `categoria` y `evidencia`; lista de tres criterios y una limitación identificada. **No equivale a un modelo de IA.**

Analogical Prompting

Idea central de la técnica

El modelo puede generar **ejemplos o conocimientos análogos relevantes** y usarlos como contexto para resolver un problema nuevo. Importa la similitud estructural, no solo el parecido superficial.

- Identificar un problema comparable y su principio transferible.
- Revisar diferencias relevantes entre dominios.
- Aplicar exclusivamente los mecanismos justificados.

Fuente: Yasunaga et al., Large Language Models as Analogical Reasoners (ICLR 2024)

Analogical: del seguimiento de pedidos a solicitudes

Elemento	Seguimiento de pedidos	Atención institucional
Entrada	Orden o incidencia.	Solicitud con identificación ficticia.
Estado	Recibido, procesando, cerrado.	Registrada, en revisión, resuelta.
Asignación	Departamento responsable.	Área académica, financiera o técnica.
Excepción	Pedido sin información.	Solicitud ambigua: revisión manual.

Límite de la analogía

El seguimiento de pedidos no prueba que las mismas reglas de prioridad sean apropiadas para trámites de estudiantes.

Step-Back y Analogical no hacen lo mismo

Técnica	Movimiento principal	Pregunta orientadora
Step-Back	Del caso a principios generales y regreso.	¿Qué marco conceptual debo usar?
Analogical	Del caso a ejemplos comparables y regreso.	¿Qué estructura de otro problema puedo transferir?

Regla práctica

Step-Back busca **abstraer**; Analogical busca **transferir**. Ambos requieren comprobar la pertinencia al caso.

Actividad 2: comparar tres enfoques

Laboratorio de Analítica de Datos - Sede Bogotá -

UNIVERSIDAD NACIONAL DE COLOMBIA

En parejas — 10 minutos

Pide al agente **tres propuestas** de mejora del clasificador: (A) prompt directo, (B) Step-Back y (C) Analogical usando una mesa de ayuda o pedidos.

- Evalúa cobertura, supuestos nuevos y tratamiento de S06.
- Si el agente genera código, exige **solo Python**.
- Conserva una mejora concreta que reutilizarás en la siguiente sección.

Idea clave

Entregable: tabla de comparación y decisión justificada; no basta con decir «la respuesta es mejor».

DataLab UNAL

Encontrar un buen marco no determina automáticamente

qué debemos hacer primero.

Nueva pregunta

¿Conviene comenzar a ejecutar inmediatamente o construir antes una estrategia verificable de resolución?

Plan-and-Solve

Idea central

Plan-and-Solve separa explícitamente **planificación** y **ejecución**: se divide la tarea, se establece una secuencia y después se realizan los subproblemas de acuerdo con el plan.

- El plan puede inspeccionarse antes de realizar trabajo costoso.
- Una lista no es un plan útil si no define dependencias y verificación.
- Pueden quedar pasos incompletos si faltan datos o permisos.

Fuente: Wang et al., Plan-and-Solve Prompting (ACL 2023)

Plan-and-Solve: cuatro momentos



¿Qué contiene un plan verificable?

Archivos por crear, orden de acciones, entradas y salidas, supuestos, casos de prueba, criterios para detener o revisar.

Idea clave

Planificar no significa empezar a resolver en forma de lista: es decidir el procedimiento antes de ejecutarlo.

Plan-and-Solve aplicado en Python

Laboratorio de Analítica de Datos - Sede Bogotá -

UNIVERSIDAD NACIONAL DE COLOMBIA

Prompt al agente, fase 1

Diseña un script Python que lea seis solicitudes ficticias de un archivo CSV, las clasifique y genere un reporte. Antes de escribir archivos, enumera tareas, entradas, salidas, dependencias y pruebas. **No programes todavía.**

Prompt al agente, fase 2

Aplica el plan aprobado. Implementa cada etapa y ejecuta pruebas para S01–S06. Si los datos no permiten resolver una categoría, utiliza «Revisión manual».

Idea clave

Un agente puede proponer un buen plan y escribir código incorrecto: las pruebas siguen siendo necesarias.

DataLab UNAL

Actividad 3: planificar antes de implementar

En parejas — 10 minutos

1) Pide un plan para procesar un CSV con los mensajes S01–S06.
2) Identifica **un requisito omitido** y solicita una corrección del plan.
3) Autoriza la implementación **solo de lectura y validación de datos**.
4) Ejecuta un archivo de prueba con una fila vacía.

Evidencia de aprendizaje

Guarda `plan.md`, la función de lectura y una prueba que indique qué ocurre si falta un mensaje.

Decomposed Prompting

Laboratorio de Analítica de Datos - Sede Bogotá -

UNIVERSIDAD NACIONAL DE COLOMBIA

Idea central

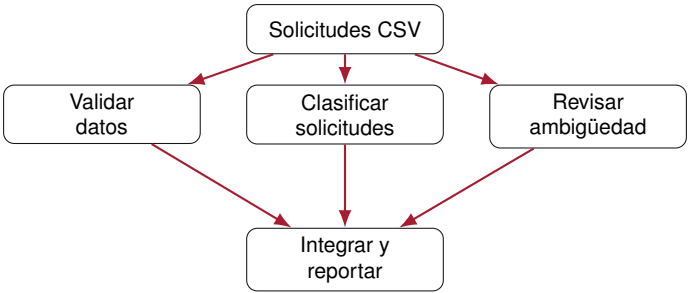
Se divide una tarea compleja en **subtareas o módulos especializados**, cada uno con su función. Los módulos pueden relacionarse, reutilizar resultados y dividirse de nuevo si hace falta.

- Diferenciar componentes y definir su interfaz de entrada/salida.
- Coordinar dependencias, no asumir independencia total.
- Integrar las respuestas y comprobar que no se propaguen errores.

Fuente: Khot et al., Decomposed Prompting (2023)

DataLab UNAL

Decomposed Prompting: módulos funcionales



Idea clave

La modularidad describe responsabilidades; no exige que las funciones se ejecuten en paralelo.

Aplicación: entradas, salidas y dependencias

Módulo	Entrada	Salida	Comprobación
Lectura	Ruta CSV.	Lista de registros.	IDs presentes.
Clasificación	Texto de solicitud.	Etiqueta + evidencia.	Casos de referencia.
Ambigüedad	Resultado provisional.	Confirmar o revisar.	S06 no forzado.
Reporte	Registros procesados.	Conteos y casos dudosos.	Totales coherentes.

Diseño coherente

La salida de un módulo debe tener la estructura que espera el siguiente; documenta esa interfaz.

Least-to-Most vs. Decomposed Prompting

Estrategia	Organización	Cuándo ayuda
Least-to-Most	Subproblemas dependientes resueltos en orden, usando resultados anteriores.	Cuando cada paso habilita el siguiente.
Decomposed Prompting	Subtareas asignadas a módulos especializados, potencialmente jerárquicos y con dependencias.	Cuando se requieren capacidades distintas, reutilizables o comprobables por separado.

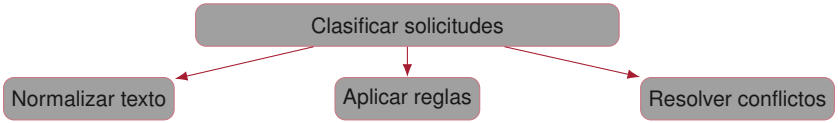
Idea clave

Ambos descomponen; difieren en el énfasis de **secuencia de resolución** frente a **diseño modular**.

Descomposición recursiva

Idea central

Si un subproblema sigue siendo demasiado amplio, puede subdividirse de nuevo hasta alcanzar una unidad con responsabilidades y pruebas claras.



Idea clave

Criterio de parada: una unidad puede implementarse y probarse de forma comprensible.

Actividad 4: construir por módulos

Con un agente de programación — 12 minutos

1) Diseña `leer_solicitudes`, `clasificar`, `revisar_ambiguedad` y `reporte`.
2) Pide implementación **solo en Python estándar** y pruebas pequeñas.
3) Ejecuta S01–S06 y corrige el manejo de mensajes vacíos y de S06.

Entregable

Código modular, prueba por función y registro de una corrección realizada tras ejecutar. **No hacer interfaz web.**

Una solución razonable no siempre es la mejor

Laboratorio de Analítica de Datos - Sede Bogotá -

UNIVERSIDAD NACIONAL DE COLOMBIA

Problema de exploración

Para una misma tarea pueden existir soluciones plausibles con distinto costo, complejidad, precisión o necesidad de datos adicionales.

- La respuesta directa puede favorecer la primera ruta propuesta.
- Explorar alternativas permite explicitar ventajas y riesgos.
- Los criterios de selección deben definirse **antes** de conocer la opción ganadora.

Idea clave

¿Cómo cambia la elección cuando obligamos al procedimiento a explorar antes de decidir?

DataLab UNAL

Tree-of-Thought (ToT)

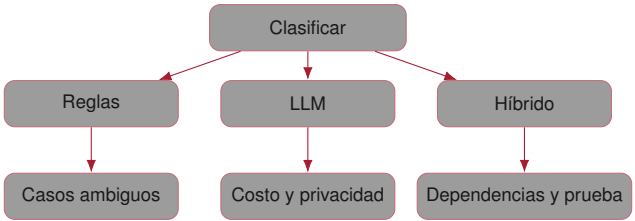
Idea central

En lugar de continuar por una sola trayectoria, se proponen **estados o rutas parciales**, se evalúan y se expanden las más prometedoras; puede haber retroceso si una ruta falla.

- Generar candidatos y definir una función/criterio de evaluación.
- Seleccionar o descartar ramas; revisar estados intermedios.
- El método añade costo y no garantiza encontrar una respuesta correcta.

Fuente: Yao et al., Tree of Thoughts (2023)

ToT: árbol de alternativas y subdecisiones



Idea clave

Las ramas no son puntuaciones reales: el árbol representa posibles decisiones que deberán contrastarse con pruebas.

Tree-of-Thought aplicado: tres arquitecturas

Ruta	Descripción	Aspectos a contrastar
A. Palabras clave	Buscar términos evidentes en el texto.	Simple; frágil frente a paráfrasis y negaciones.
B. Reglas estructuradas	Reglas y prioridad explícita ante conflictos.	Auditabilidad; mantenimiento y cobertura.
C. LLM	Clasificar con un modelo externo o local.	Mayor flexibilidad potencial; costo, incertidumbre y privacidad.

Regla metodológica

No atribuir exactitud a una ruta sin evaluarla con ejemplos anotados y no vistos durante su diseño.

Evaluar ramas antes de continuar

Criterio	Pregunta de evaluación
Factibilidad	¿Puede implementarse con los recursos disponibles?
Costo	¿Qué gastos, dependencias y uso de cómputo implica?
Alcance	¿Qué variedad de mensajes puede tratar?
Tiempo	¿Se puede ejecutar y probar durante el taller?
Evidencia disponible	¿Qué resultados observables respaldan esta decisión?

Idea clave

Las puntuaciones del agente son opiniones iniciales, no mediciones; deben diferenciarse de resultados obtenidos al ejecutar pruebas.

Explorar en varios niveles

Un ejemplo de evaluación con retroceso

1. Se elige inicialmente **reglas simples** por su bajo costo.
2. Se observa que un mensaje contiene términos de dos áreas.
3. Se expande la rama: «priorizar», «pedir información» o «revisión manual».
4. Se selecciona revisión manual si la evidencia no justifica una etiqueta única.

Qué documentar

Estado inicial, ramas consideradas, criterios, pruebas realizadas, decisión y razón de un eventual retroceso.

Idea clave

La elección debe poder revisarse con datos nuevos, no mantenerse por inercia.

Actividad 5: Tree-of-Thought

Con un agente — 12 minutos

1) Solicita tres rutas de clasificación de S01–S06.
2) Define previamente factibilidad, explicabilidad y manejo de ambigüedad.
3) Descarta una alternativa por una restricción justificada.
4) Pide una prueba en **Python** para las dos restantes.
5) Selecciona o reconsidera según los resultados.

Producto

Tabla de criterios, árbol de alternativas y evidencia de al menos una ejecución.

Una ruta elegida debe verificarse

Evaluación previa

- Costo estimado.
- Dependencias previstas.
- Explicabilidad esperada.

Evaluación posterior

- Clasificaciones observadas.
- Errores detectados.
- Casos de revisión manual.

Idea clave

Una descripción convincente o el consenso entre varias respuestas no demuestra que una solución funcione.

¿Qué aprendimos de las alternativas?

Preguntas para cerrar la exploración

- ¿Qué decisión cambió tras examinar otra ruta?
- ¿Qué incertidumbres seguían sin resolverse?
- ¿Qué información nueva haría cambiar la elección?
- ¿Qué alternativa conserva mejor la trazabilidad?

Conclusión

ToT organiza la búsqueda de soluciones; no reemplaza el diseño de pruebas ni las limitaciones del modelo.

Skeleton-of-Thought (SoT)

Idea central

Primero se genera un **esqueleto mínimo de una respuesta**; después se expanden sus puntos. La propuesta académica explora la expansión **paralela** para reducir latencia.

- El esqueleto define cobertura y orden de comunicación.
- Puede desarrollarse secuencialmente como adaptación pedagógica.
- No debe confundirse con explorar rutas de solución (ToT).

Fuente: Ning et al., Skeleton-of-Thought (ICLR 2024)

Skeleton-of-Thought aplicado a un informe

Laboratorio de Analítica de Datos - Sede Bogotá -

UNIVERSIDAD NACIONAL DE COLOMBIA

Fase 1: producir únicamente el esqueleto

1. Resumen de solicitudes; 2. Distribución por categoría; 3. Casos ambiguos; 4. Recomendaciones; 5. Limitaciones de datos y del clasificador.

Fase 2: desarrollar apartados

Completa cada sección utilizando **solo resultados obtenidos** de S01–S06. Separa observaciones, recomendaciones y datos faltantes; no inventes causas ni tasas generales.

Idea clave

Un informe bien estructurado puede estar equivocado: además se deben revisar sus afirmaciones.

DataLab UNAL

Tree-of-Thought vs. Skeleton-of-Thought

Técnica	Qué organiza	Pregunta principal
Tree-of-Thought	Alternativas, estados y decisiones de resolución.	¿Qué ruta debería probar o desarrollar?
Skeleton-of-Thought	Esquema y desarrollo de una respuesta.	¿Cómo estructurar y completar lo que comunicaré?

Lectura práctica

ToT trabaja sobre el **espacio de soluciones**; SoT sobre la **estructura de la salida**. Se pueden combinar, pero tienen objetivos diferentes.

Actividad 6: informe en Python

Con un agente — 10 minutos

1) A partir de los resultados anteriores, solicita un **esqueleto** de informe.
2) Revisa si cubre resultados, casos ambiguos y limitaciones.
3) Pide una función Python que genere `informe.txt` o `informe.md`.
4) Verifica que los conteos sumen seis y que no se atribuyan causas inexistentes.

Entregable

Archivo de texto generado, estructura explícita y una verificación numérica simple.

¿Qué estrategia utilizar?

Necesidad	Estrategia	Producto en Python
Encontrar principios relevantes	Step-Back	Criterios de diseño
Transferir un patrón comparable	Analogical	Estados y excepciones
Decidir el procedimiento antes de ejecutar	Plan-and-Solve	Plan de tareas
Separar funciones	Decomposed	Módulos y tests
Profundizar un subproblema	Recursiva	Funciones más pequeñas
Explorar opciones	Tree-of-Thought	Comparación y pruebas
Organizar una respuesta	Skeleton-of-Thought	Informe estructurado

Idea clave

No se elige la técnica por moda: la estructura del problema, el riesgo y la verificación determinan su utilidad.

Reto final: chat con modo Thinking

Objetivo

Construir **en Python y desde la terminal** un chat de consulta de solicitudes, con dos modos: **Rápido** (una respuesta) y **Thinking** (tres etapas externas y observables).

- Un agente de programación ayuda a construir, ejecutar y corregir el código.
- Para responder como chat de IA, el script debe conectarse a un LLM accesible.
- No se utilizarán HTML, CSS, JavaScript, Streamlit ni aplicaciones web.

Alcance realista

Prototipo de consola con mensajes ficticios, manejo de errores y pruebas, no un sistema institucional listo para producción.

Arquitectura conceptual del chat



Modo rápido

Una llamada al modelo; respuesta breve, con indicación de incertidumbre cuando corresponda.

Modo Thinking

Tres llamadas organizadas, guardando un resumen verificable de cada etapa.

Idea clave

La lógica de etapas pertenece al programa; no se asume acceso al razonamiento interno del modelo.

Modo Thinking: tres iteraciones externas

Etapa	Técnicas de apoyo	Salida observable
1. Comprender	Step-Back y Analogical	Dimensiones, datos disponibles, supuestos y datos faltantes.
2. Explorar	Plan-and-Solve, Decomposed y ToT	Opciones comparables con criterios y dependencias.
3. Sintetizar	Skeleton y verificación	Respuesta estructurada, recomendación condicionada y límites.

Rigor conceptual

No se imprime ni se afirma reconstruir la cadena privada de pensamiento del LLM: se muestran **productos explícitos** de etapas distintas.

Paso 1: Step-Back y Plan-and-Solve

Prompt para el agente de código: antes de programar

Quiero un chat de terminal en Python con modos Rápido y Thinking. Antes de escribir archivos, identifica principios para responder solicitudes sin inventar información (Step-Back) y presenta un plan con módulos, dependencias, pruebas y criterios de aceptación (Plan-and-Solve). **No programes todavía.**

Revisión del estudiante

¿El plan distingue correctamente el **agente que programa** del **LLM que responderá**? ¿Incluye errores de conexión y una forma de terminar la conversación?

Paso 2: dividir las responsabilidades

Laboratorio de Analítica de Datos - Sede Bogotá - UNIVERSIDAD NACIONAL DE COLOMBIA

Prompt de descomposición

Tras aprobar el plan, divide el proyecto en módulos y define la firma de las funciones. Implementa por partes, usando Python y pruebas para cada función.

Archivo sugerido	Responsabilidad
`chat.py`	Leer entrada, seleccionar modo y cerrar la sesión.
`modelo.py`	Enviar solicitudes al LLM, gestionar tiempo de espera y errores.
`estrategias.py`	Ejecutar una o tres etapas y construir el mensaje final.
`test_chat.py`	Probar flujo, entradas y respuestas simuladas sin llamar a la red.

Idea clave

Estos archivos son una opción didáctica, no una arquitectura obligatoria.

DataLab UNAL

Paso 3: explorar implementaciones viables

Prompt Tree-of-Thought para el agente

Compara tres opciones para conectar el chat: (A) Ollama local; (B) API remota con credenciales; (C) modo de prueba sin LLM real. Evalúa configuración, dependencia de red, privacidad y capacidad de ejecutar el taller. No inventes tiempos o costos. Recomienda la opción que pueda probarse con los recursos disponibles.

Criterio de verdad

Si no hay servidor o credenciales, un **modo simulado** es válido para probar el flujo, pero no debe presentarse como respuesta producida por un LLM.

Paso 4: implementar el chat desde la terminal

Laboratorio de Analítica de Datos – Sede Bogotá

UNIVERSIDAD NACIONAL DE COLOMBIA

Prompt de implementación — 20 minutos

Implementa el plan aprobado en Python. Pide el modo (1=Rápido, 2=Thinking); acepta preguntas en un bucle; usa un modelo configurable; en Thinking realiza tres llamadas con prompts distintos; conserva únicamente resúmenes legibles de cada etapa; permite escribir «salir». Gestiona entradas vacías, fallos de conexión y límites de contexto. No imprimas claves ni información privada.

Ejecutar y revisar

```
python chat.py    /    python -m unittest -v
```

Idea clave

No exigir interfaces web ni una implementación completa de un agente autónomo.

DataLab UNAL

Pruebas y discusión del prototipo

Prueba	Criterio verificable	Puntos
Modo rápido	Una llamada y respuesta breve.	0–2
Modo Thinking	Tres etapas externas diferenciables.	0–2
Robustez	Gestiona pregunta vacía, error de conexión y «salir».	0–2
Fidelidad	Señala incertidumbre y no inventa datos institucionales.	0–2
Evidencia	Explica plan, módulos, elección y pruebas realizadas.	0–2
Total	**Máximo**	**10**

Discusión final

¿Qué mejoró con la planificación, la descomposición y la evaluación? ¿Qué problemas permanecen sin resolver?

Referencias y cierre

- Zheng et al. (2023). *Take a Step Back: Evoking Reasoning via Abstraction in LLMs*. arXiv:2310.06117.
- Yasunaga et al. (2024). *Large Language Models as Analogical Reasoners*. ICLR / arXiv:2310.01714.
- Wang et al. (2023). *Plan-and-Solve Prompting*. ACL 2023.
- Khot et al. (2023). *Decomposed Prompting*. arXiv:2210.02406.
- Zhou et al. (2023). *Least-to-Most Prompting*. arXiv:2205.10625.
- Yao et al. (2023). *Tree of Thoughts*. arXiv:2305.10601.
- Ning et al. (2024). *Skeleton-of-Thought*. ICLR / arXiv:2307.15337.

Mensaje final

Definir la tarea, escoger una estrategia, ejecutar y verificar: el valor de un prompt se observa en **resultados comprobables**.